Algoritma Analizi Dönem Projesi Raporu

MUSTAFA TALHA KARAGENÇ

231201012

ÖZET

Bu proje çalışmasında, bilgisayar bilimlerinin temel taşlarından olan beş farklı sıralama algoritması (Bubble Sort, Insertion Sort, Merge Sort, Heap Sort ve Quick Sort) incelenmiştir. Çalışmanın amacı, bu algoritmaların teorik zaman karmaşıklıklarını (Time Complexity) deneysel verilerle karşılaştırmak ve hangi algoritmanın hangi veri senaryosunda daha verimli olduğunu tespit etmektir. Python programlama dili kullanılarak geliştirilen simülasyon ortamında, algoritmalar "Sıralı", "Rastgele" ve "Ters Sıralı" olmak üzere üç farklı senaryoda test edilmiştir. Elde edilen grafiksel veriler ve analizler sonucunda, küçük ve sıralı veri setlerinde basit algoritmaların (Insertion Sort), büyük ve karmaşık veri setlerinde ise logaritmik karmaşıklığa sahip algoritmaların (Quick Sort, Merge Sort) daha üstün performans gösterdiği doğrulanmıştır.

GİRİŞ

Sıralama (Sorting), verilerin belirli bir kurala göre (küçükten büyüğe veya tam tersi) düzenlenmesi işlemidir ve veritabanı yönetiminden arama motorlarına kadar birçok alanda kritik öneme sahiptir. Bir algoritmanın verimliliği genellikle **Big-O (Büyük-O)** notasyonu ile ifade edilen zaman ve alan karmaşıklığı ile ölçülür.

Bu projenin amacı:

1. Farklı çalışma prensiplerine sahip (karşılaştırmalı, böl-ve-fethet vb.) algoritmaları kodlayarak yapılarını anlamak.
2. Teorik olarak hesaplanan On^2 ve O(n log n) karmaşıklıklarının gerçek çalışma sürelerine etkisini gözlemlemek.
3. Veri boyutu arttıkça algoritmaların performans değişimlerini analiz etmektir.

YÖNTEM

ullanılan Araçlar ve Ortam

- Yazılım Dili: Python 3.12
- Kütüphaneler**:**
  - time: Algoritmaların milisaniye cinsinden çalışma sürelerini ölçmek için.
  - random: Test verilerini oluşturmak için.
  - matplotlib: Sonuçları görselleştirmek ve performans grafikleri çizmek için.
  - sys: Rekürsif algoritmalar (Quick ve Merge Sort) için sistemin özyineleme limitini artırmak amacıyla (sys.setrecursionlimit) kullanılmıştır

Bubble Sort

- Çalışma Mantığı: Liste üzerinde gezinirken komşu elemanları karşılaştırır ve yanlış sıradaysa yer değiştirir (swap). Bu işlem, liste sıralanana kadar tekrarlanır.
- Kod Analizi: Kodda swapped bayrağı (flag) kullanılarak optimizasyon yapılmıştır. Eğer bir turda hiç yer değiştirme olmazsa döngü kırılır.
- Zaman Karmaşıklığı:
  - Best Case: O(n)(Liste zaten sıralıysa).
  - Average/Worst Case: On²
- Alan Karmaşıklığı: O(1) (Ekstra bellek kullanmaz).

```python
def bubble_sort(arr):
    n = len(arr)
    # Optimize edilmiş Bubble Sort (Best Case O(n) için)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
```

Insertion Sort (Araya Ekleme Sıralaması)

- **Çalışma Mantığı:** Dizinin ikinci elemanından başlayarak, seçilen elemanı sol tarafındaki sıralı alt dizi içinde doğru konuma yerleştirir. İskambil kağıtlarını elimizde dizer gibi çalışır.

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

Merge Sort (Birleştirmeli Sıralama)

- **Çalışma Mantığı:** "Böl ve Fethet" (Divide and Conquer) yöntemini kullanır. Diziyi sürekli ikiye böler, en küçük parçaları sıralar ve ardından sıralı parçaları birleştirerek (merge) ana diziyi oluşturur.

```python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
```

Heap Sort (Yığın Sıralaması)

- **Çalışma Mantığı:** Diziyi önce bir Binary Heap (Max-Heap) veri yapısına dönüştürür. Kök düğümdeki en büyük elemanı dizinin sonuna atar ve heap yapısını yeniden düzenler (heapify).

```python
def heap_sort(arr):
    def heapify(arr, n, i):
        largest = i
        l = 2 * i + 1
        r = 2 * i + 2

        if l < n and arr[i] < arr[l]:
            largest = l
        if r < n and arr[largest] < arr[r]:
            largest = r
        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(arr, n, largest)

    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
```

Quick Sort (Hızlı Sıralama)

- **Çalışma Mantığı:** Bir "Pivot" eleman seçer. Pivot'tan küçükleri sola, büyükleri sağa atarak diziyi parçalar (Partitioning). Bu işlem recursive olarak devam eder.

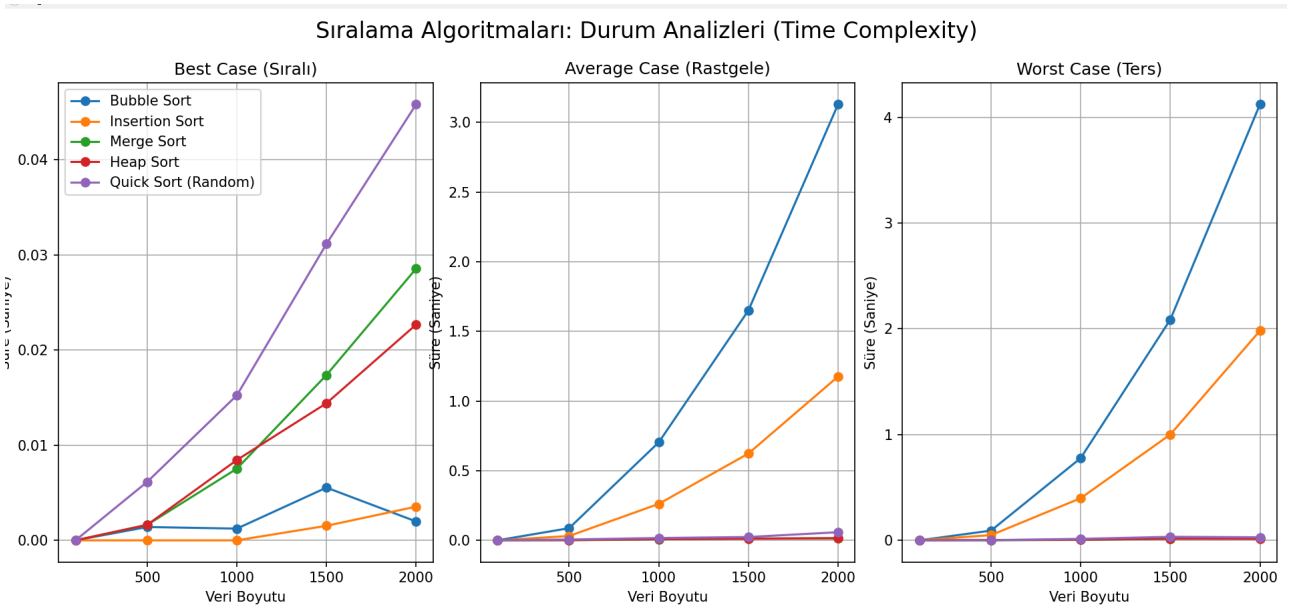
```python
# Quick Sort (Random Pivot )
def quick_sort(arr, low, high):
    if low < high:
        rand_idx = random.randint(low, high)
        arr[rand_idx], arr[high] = arr[high], arr[rand_idx]
        pivot = arr[high]
        i = low - 1
        for j in range(low, high):
            if arr[j] <= pivot:
                i = i + 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        pi = i + 1
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def run_quick_sort(arr):
    quick_sort(arr, 0, len(arr) - 1)
```

Veri Seti:

- **Veri Aralığı:** 0 ile 10.000 arasındaki tam sayılar.
- **Veri Boyutları:** 100, 500, 1000, 1500 ve 2000 elemanlı diziler.
- **Senaryolar:**
  1. **Sorted (Best Case):** Küçükten büyüğe sıralı veri.
  2. **Random (Average Case):** Rastgele dağılmış veri.
  3. **Reverse (Worst Case):** Büyükten küçüğe ters sıralı veri.

Uygulama Yöntemi

Yazılım, modüler bir yapıda tasarlanmıştır. run_tests_and_analyze fonksiyonu, belirlenen 5 farklı veri boyutunda ve 3 farklı senaryoda tüm algoritmaları sırayla çalıştırır. Her algoritmanın başlangıç ve bitiş zamanları (time.time()) kaydedilir ve aradaki fark performans skoru olarak saklanır. Son olarak matplotlib kütüphanesi ile 3 ayrı grafik (Sıralı, Rastgele, Ters) çizdirilir.



SONUÇ

Yapılan bu çalışma, algoritmaların performansının sadece kullanılan koda değil, işlenen verinin yapısına ve boyutuna doğrudan bağlı olduğunu göstermiştir.

Elde edilen sonuçlara göre:

- Veri seti küçükse veya büyük oranda sıralıysa **Insertion Sort** tercih edilmelidir.
- Genel amaçlı, büyük ve karışık veri setlerinde **Quick Sort** veya **Merge Sort** kullanılmalıdır.
- Bellek kullanımının kritik olduğu (Space Complexity kısıtı) büyük veri setlerinde **Heap Sort** iyi bir alternatiftir.

Proje, teorik olarak derslerde işlenen Big-O analizlerinin pratik dünyadaki karşılığını somut verilerle doğrulamış ve algoritma seçiminin yazılım performansındaki kritik rolünü ortaya koymuştur.

```
BELLEK KULLANIMI (Peak Memory - Max Data Size):
- Bubble Sort         : 0.30 KB
- Insertion Sort      : 0.18 KB
- Merge Sort          : 31.34 KB
- Heap Sort           : 0.58 KB
- Quick Sort (Random) : 3.96 KB
```

KAYNAKÇA

- Kod dünyası web youtube
- Python Software Foundation. (2024). *Python 3.12 Documentation*.
- kavramharitası web youtube
- ders kaynakları